

Rapport

Mai 2025

Contexte & objectifs du sprint

Le projet est un script Python qui tourne en fond de tâche sur un PC de cabinet dentaire. Quand un appareil d'imagerie dépose un fichier, le script l'identifie, le déplace dans le bon dossier patient sur le réseau, et insère l'entrée dans la base Access de StudioVision.

Objectifs V4 : fiabiliser le comportement sur partage réseau, sécuriser les appels COM, et rendre le service visible/contrôlable par l'opérateur.

Ce qui a été livré

Architecture & robustesse réseau

- Remplacement du `Observer` classique par un `PollingObserver` — plus lent mais fiable à 100 % sur les chemins UNC.
- Boucle d'auto-reconnexion : si le partage réseau disparaît, le thread se met en attente et relance l'observer dès que le dossier est de nouveau accessible.
- Pattern *Producer/Consumer* avec **burst debounce** à 1,5 s : un seul rafraîchissement COM par lot d'images, même si dix fichiers arrivent en rafale.

Sécurisation des appels COM (Access / StudioVision)

- **Dirty-state guard** : vérification de `form.Dirty` avant tout `Requery()` pour éviter qu'Access ne lève une boîte de dialogue de sauvegarde.
- **Requery retry loop** : 3 tentatives espacées de 0,5 s, avec fallback sur `Refresh()` en dernier recours.
- **Patient-code guard** : si l'opérateur a changé de dossier patient pendant le debounce, le rafraîchissement de l'UI est annulé.

Expérience utilisateur (UX)

- **Icône System Tray** (via `pystray` + `Pillow`) tournant sur le thread principal. Code couleur : **Bleu** = en attente, **Vert** = transfert en cours. Menu contextuel : statut, accès aux logs, quitter.
- **Notifications Toast Windows** non bloquantes (3 s) : début de burst, fin de transfert (avec compteur), erreur (fichier verrouillé, orphelin).
- Logs centralisés dans `~/studiovision/image_router.log`.

Builds spécifiques

- **Box 2 Nidek OCT** : logique de déduplication par dossier de scan. Attente de 2 s de stabilisation, suppression des `.xml`, conservation de l'image la plus lourde uniquement (haute résolution), nettoyage des miniatures et des dossiers vides. Gestion des fichiers résiduels qui arrivent en retard dans la file.
- **Windows 7** : rétroport Python 3.9 (`typing.Optional` , `.format()` , `daemon=True`).

Déploiement & système

- Prévention de la mise en veille via `SetThreadExecutionState` (`ctypes.windll.kernel32`).
- Compilation en exécutable autonome (`PyInstaller -onefile -noconsole`).
- Déploiement automatisé : scripts Batch/PowerShell pour la création de raccourcis et les paramètres d'alimentation.

Livrables

Fichier	Version	Cible
studiovision_monitorV4.py	V4	Version de base
windows7V4.py	V4	Python 3.9 / Windows 7
box1V4.py	V4	Appareil standard (Box 1)
box2V4.py	V4	Nidek OCT (Box 2)

- Le `PollingObserver` interroge le dossier toutes les secondes à surveiller sur des volumes très actifs.
- La logique Nidek repose sur un délai fixe de 2 s ; si l'appareil est lent sur le réseau, ce délai pourrait être insuffisant.
- Pas de tests automatisés en place : les validations sont manuelles sur poste de test.